

보안 스터디

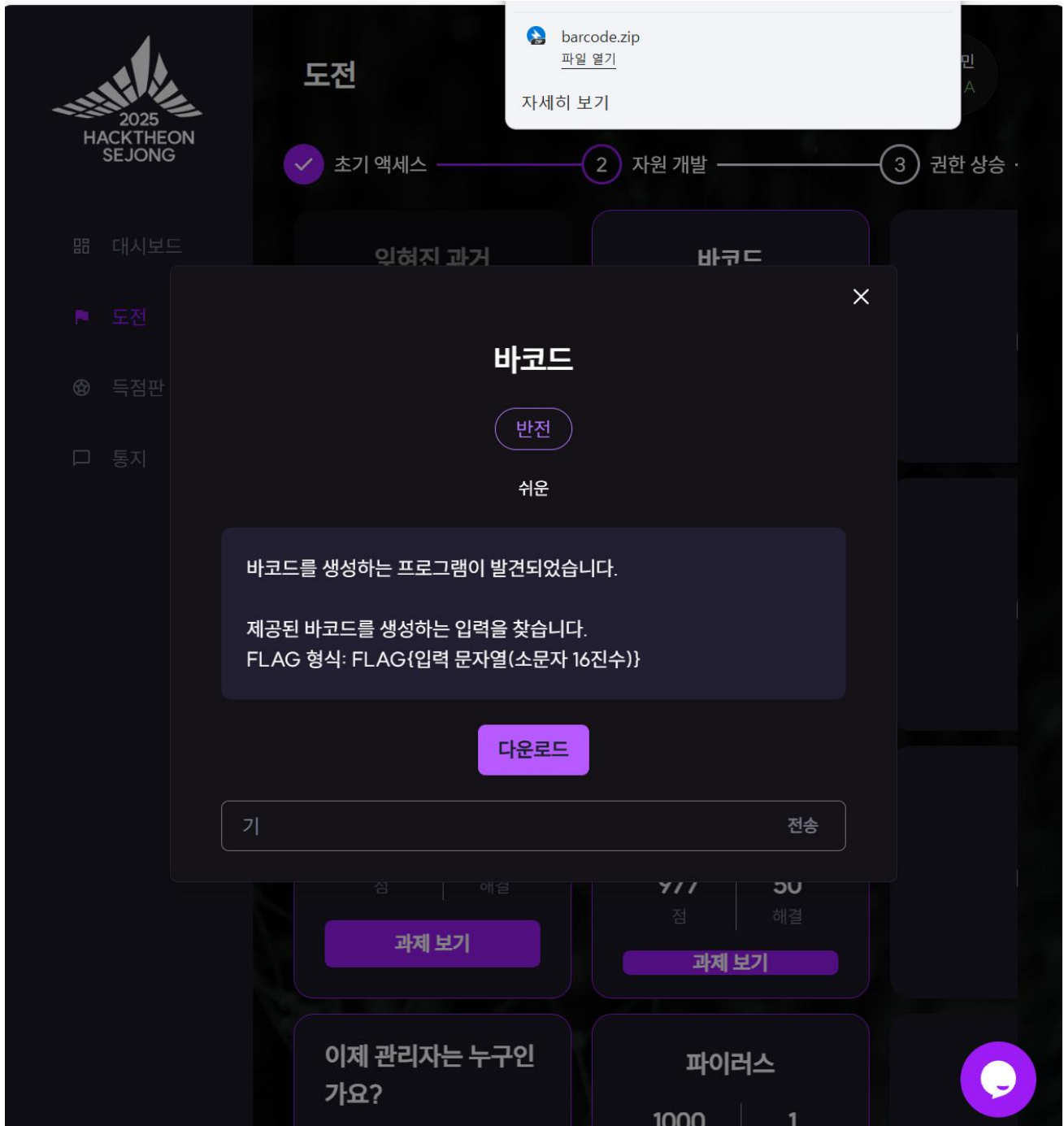
4 차시 주제: Barcode

Barcode

목차

1. 문제 설명	2
2. 접근 방법	4

1. 문제 설명



도구(T) 도움말(H)			
포함 ▾ 공유 대상 ▾ 새 폴더			
이름	수정한 날짜	유형	크기
 barcode	2025-04-03 오후 3...	파일	19KB
 flag.barcode	2025-04-21 오후 1...	BARCODE 파일	1KB

```

barcode >  flag.barcode
1
2  *****
3  *
4  *****
5  *
6  *
7  *
8
9
10 *
11 *
12 *
13 *
14 *
15 *****
16
17
18  ****
19  *      *
20  *****
21  *      *
22  *      *
23  *      *
24
25
26  ****
27  *      *
28  *
29  *      ***
30  *      *
31  ****
32
33

```

세종 핵테온에서 나온 문제 중 하나인 barcode 문제입니다.

barcode 파일의 출력 결과로 flag.barcode 파일의 문자열이 출력되게끔 하는 입력 문자열을 찾는 리버싱 문제였습니다.

2. 접근 방법

```
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode
Usage: ./barcode {hex-string}
Example: ./barcode 0x3E08080818381808
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$
```

1) 일단 프로그램을 실행해보았다.

```
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode
Usage: ./barcode {hex-string}
Example: ./barcode 0x3E08080818381808
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode 0x3E08080818381808
**
***
**
*
*
****
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$
```

2) 예시로 준 hex 값을 입력하여 실행해 보았다.

```
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode f
Invalid hex-string
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode ff
Invalid hex-string
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode fff
****

isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode ffff
*****

isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode fffff
*****
****

isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode ffffff
*****
*****

isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode ffffffff
*****
*****
****

isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$
```

- 3) 잘 모르겠어서 그냥 마구잡이로 넣어보았다.
(여기서 알 수 있었던 것은 적어도 hex 값에 두 글자 이상은 써야된다는 것과 다섯 자리 이상 입력할 경우에는 두 자리의 hex 값마다 2 진법으로 한 줄씩 출력이 이루어짐을 알 수 있었다.)

```
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode 0000ff
*****

isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode ffff00
*****

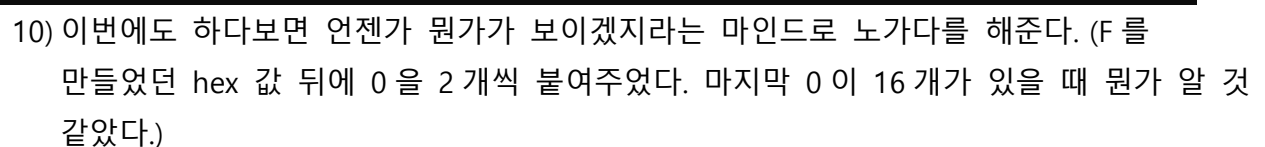
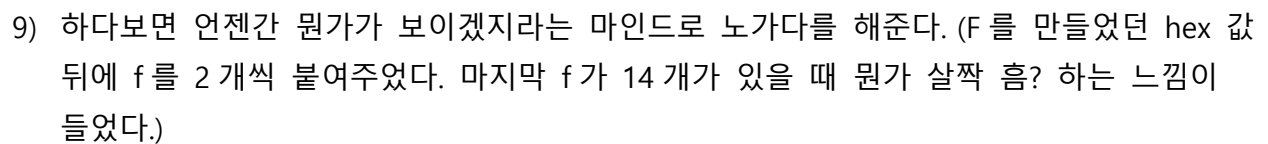
isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode ff0000

isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode aaff00
*****

isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$ ./barcode aa00ff
*****

isemy@WIN-L7B050ANG22: /mnt/c/Users/isemy/Desktop/barcode$
```

- 4) 다섯 번째 hex 값 부터는 대충 규칙을 알겠는데 앞에 네번째 hex 값까지는 규칙을 모르겠어서 여러 숫자들을 바꾸어가며 넣어보았다.
(여기서 알 수 있었던 것은 맨 앞의 두 자리 hex 값은 무시된다는 것이다.)



한 스펀링 당 16 자의 hex 값이 필요하고 F 를 작성하고 나면 L 의 자리에 아무것도 작성되지 않은 값인 0000000000000000 이 바로 윗 글자가 작성된 곳은 비고 비워져있는 곳은 작성되는 상태로 변한다는 것을 알 수 있었다.

```
isemy@WIN-L78050ANG22:/mnt/c/Users/isemy/Desktop/barcode$ ./barcode 00000202027e027e00ff83ffff83ff83ff

*****
*
*****
*
*
*
*
*
*
*****
```

11) 위에서 찾아낸 규칙을 바탕으로 FL 까지 작성했다.

```
isemy@WIN-L78050ANG22:/mnt/c/Users/isemy/Desktop/barcode$ ./barcode 00000202027e027e00ff83ffff83ff0000000000000000

*****
*
*****
*
*
*
*
*
*
*****

*****
*****

*****

isemy@WIN-L78050ANG22:/mnt/c/Users/isemy/Desktop/barcode$ ./barcode 00000202027e027e00ff83ffff83ff0000000000000000

*****
*
*****
*
*
*
*
*
*
*****

*****
*****

*****

isemy@WIN-L78050ANG22:/mnt/c/Users/isemy/Desktop/barcode$ |
```

12) A 를 작성하기 위해 16 개의 hex 값을 f와 0 으로 각각 채워주었더니 어떤 게 보였다. (F 와 L 을 겹쳐서 둘 다 있으면 지우고 하나만 있으면 남기는 식으로 하는 게 기본형(0000000000000000)이라는 것을 깨달았다.)

```
isemy@WIN-L78050ANG22:/mnt/c/Users/isemy/Desktop/barcode$ ./barcode 00000202027e027e00ff83ffff83ff003e424202420000

*****
*
*****
*
*
*
*
*
*
*****

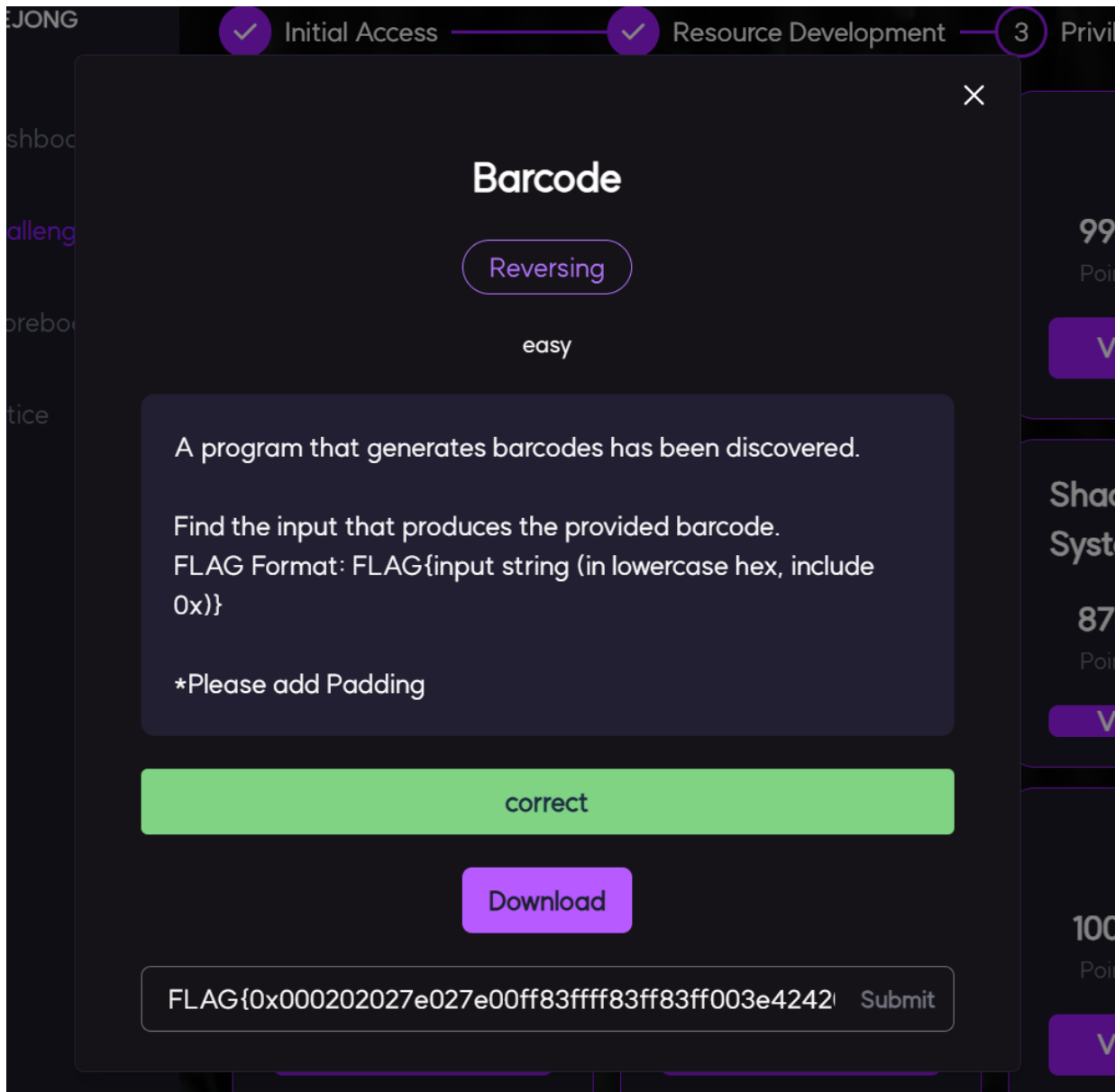
*****
*****

*****
```

13) 위에서 찾아낸 규칙을 바탕으로 FLA 를 작성했다.

16) 내가 작성한 답이 맞는지 확인하기 위해 출력된 글자를 복사해서 새로운 파일을 만든다.

17) flag.barcode 파일과 내 my.barcode 파일의 내용이 같은지 확인하기 위해 명령어를 작성해서 비교한다.



- 18) (FLAG{0x000202027e027e00ff83ffff83ff83ff003e424202424000ffdfcffff83ff}) 예시처럼 hex 값 맨 앞 두자리는 16 진수 접두어로 변경하고 플래그 형식으로 잘 작성해서 답을 입력하면 된다.